FMX Academy

| INFORMAÇÕES | | | |
|---|---|---|---|
| **MÓDULO** | | **AULA** | **INSTRUTOR** |
| **07** | **ASP.NET Core Web API – Avançado - Padrões de Projeto, Arquitetura Limpa e Alta Performance** | **02 - PUT, Soft Delete e Performance em Domínios Ricos** | **Guilherme Paracatu** |

## 1. Intencionalidade e Performance

Nesta aula, o foco é a **manutenção do estado**. No dia a dia do desenvolvimento de software para ambientes críticos, não alteramos dados "porque sim" ou simplesmente "setando" uma coluna no banco. Cada alteração no sistema deve representar uma **Intenção de Negócio**.

**A. Invariantes vs. Transição de Estado**

Precisamos separar o momento em que o objeto nasce do momento em que ele é atualizado:

- **Invariantes (O Construtor):** É a "Certidão de Nascimento". O objeto garante que todas as regras básicas para ele existir sejam cumpridas logo de cara.

- **Transição de Estado (O método de Atualização):** É a "Renovação da CNH". O objeto já existe, mas sofre uma ação específica (ex: veiculo.AlterarPlaca() em vez de veiculo.Placa = "ABC").

**B. Idempotência e o "Silêncio Produtivo"**

**Idempotência significa "ter o mesmo efeito".**

Em programação e APIs REST, uma operação é considerada idempotente se executá-la **uma única vez** tiver exatamente o mesmo efeito no sistema do que executá-la **mil vezes seguidas**.

Uma API profissional deve ser **Idempotente**. No método HTTP PUT, se o usuário enviar exatamente o que já existe no banco, o sistema deve responder "200 OK" sem realizar processamento desnecessário, e sem “xingar” o usuário com um erro.

💡 **Vantagem Técnica:** Usando a técnica do "Silêncio Produtivo" (um return silencioso), o Entity Framework Core não marca o objeto como "sujo" (*dirty*). Isso evita um comando UPDATE inútil no SQL Server, poupando processamento e rede.

**C. O Problema do "Big Data" em Relacionamentos**

Ao validar regras que envolvem entidades "filhas" (ex: "Não desativar uma Marca que possui Veículos ativos"), a primeira intuição do desenvolvedor é usar o .Include() para trazer tudo e fazer um if.

- **O Risco:** E se uma marca tiver 50.000 veículos? O servidor vai alocar memória para 50.000 objetos apenas para rodar um simples if. O sistema vai travar.

- **A Solução (Injeção de Fatos):** O Handler pergunta ao banco de dados e recebe apenas uma resposta booleana otimizada (Sim/Não). Ele entrega essa "certidão" para a Entidade decidir. Chamamos isso de entregar o *fato* ao *domínio*.

**D. Agregados e a Criação de Dados**
É comum a dúvida: *"Devo criar a Marca e os Veículos no mesmo endpoint POST?"* A resposta está na regra de negócio. Se uma Marca pode existir sem veículos no sistema (são cadastros independentes), eles são **Agregados separados**. Devemos ter endpoints granulares: cria-se a Marca primeiro e, com o Id dela, cadastram-se os Veículos.

## 2. Implementação

Agora vamos traduzir toda essa teoria para o nosso código C#. Note como as classes concentram toda a inteligência e validação.

**📖 Prática Guiada: O Ciclo de Vida do Veículo**
Nesta seção, vamos implementar o fluxo completo de atualização (PUT) e exclusão lógica (DELETE) para a entidade Veiculo. Nossa arquitetura utilizará a técnica de **Injeção de Fatos**: o banco de dados e os Handlers apenas levantam as informações, mas é a **Entidade de Domínio** quem toma a decisão final e lança as exceções.

**Passo 1: O Domínio Rico (O Juiz das Regras)**
A nossa entidade recebe os "fatos" do mundo externo (como placaJaExisteEmOutroVeiculo) e aplica as validações de negócio, garantindo que o estado do objeto seja sempre válido.

**📄 ITB.Domain/Entities/Veiculo.cs**
***Código C#***
```
using System;
using ITB.Domain.Core.Exceptions;

namespace ITB.Domain.Entities;

public class Veiculo
{
    public int Id { get; private set; }
    public string Modelo { get; private set; } = null!;
    public string Placa { get; private set; } = null!;
    public int Ano { get; private set; }
    public int MarcaId { get; private set; }
    public bool Ativo { get; private set; } = true;

    public virtual Marca Marca { get; private set; } = null!;

    protected Veiculo() { }
```

```
public Veiculo(string modelo, string placa, int ano, int marcaId)
{
    ValidarPlaca(placa);
    ValidarDados(modelo, ano);

    if (marcaId <= 0) throw new DomainException("A Marca é obrigatória.");

    Modelo = modelo;
    Placa = placa;
    Ano = ano;
    MarcaId = marcaId;
}

// --- Comportamentos de Atualização (PUT) ---

public void AtualizarDados(string modelo, int ano)
{
    if (Modelo == modelo && Ano == ano) return; // Idempotência
    ValidarDados(modelo, ano);
    Modelo = modelo;
    Ano = ano;
}

// Recebe o FATO do banco de dados para tomar a decisão
public void AlterarPlaca(string novaPlaca, bool placaJaExisteEmOutroVeiculo)
{
    if (Placa == novaPlaca) return; // Idempotência

    if (placaJaExisteEmOutroVeiculo)
        throw new DomainException("Esta placa já pertence a outro veículo.");

    ValidarPlaca(novaPlaca);
    Placa = novaPlaca;
}

// Recebe o FATO do banco de dados para tomar a decisão
public void AlterarMarca(int novaMarcaId, bool novaMarcaExiste)
{
    if (MarcaId == novaMarcaId) return; // Idempotência

    if (novaMarcaId <= 0)
        throw new DomainException("A nova marca informada é inválida.");

    if (!novaMarcaExiste)
        throw new DomainException("A nova marca informada não existe no sistema.");

    MarcaId = novaMarcaId;
}
```

```
    // --- Comportamento de Exclusão (DELETE) ---

    public void Desativar()
    {
        if (!Ativo) return; // Idempotência
        Ativo = false;
    }

    // --- Validações Internas ---

    private void ValidarPlaca(string placa)
    {
        if (string.IsNullOrWhiteSpace(placa) || placa.Length != 7)
            throw new DomainException("Formato de placa inválido.");
    }

    private void ValidarDados(string modelo, int ano)
    {
        if (string.IsNullOrWhiteSpace(modelo)) throw new DomainException("Modelo
obrigatório.");
        if (ano < 1900 || ano > DateTime.Now.Year + 1) throw new DomainException("Ano
inválido.");
    }
}
```

**ITB.Domain/Interfaces/IVeiculoRepository.cs**

***Código C#***

```
using System.Collections.Generic;
using System.Threading.Tasks;
using ITB.Domain.Entities;

namespace ITB.Domain.Interfaces;

public interface IVeiculoRepository
{
    Task Adicionar(Veiculo veiculo);
    Task Atualizar(Veiculo veiculo);
    Task<IEnumerable<Veiculo>> ObterTodos();
    Task<Veiculo?> ObterPorId(int id);
    Task<bool> PlacaJaExiste(string placa, int veiculoIdIgnorado);
}
```

**Passo 2: A Infraestrutura (O Banco de Dados)**

**ITB.Infrastructure/Repositories/VeiculoRepository.cs**

***Código C#***

```
using System.Collections.Generic;
using System.Threading.Tasks;
using Microsoft.EntityFrameworkCore;
```

```
using ITB.Domain.Entities;
using ITB.Domain.Interfaces;
using ITB.Infrastructure.Persistence;

namespace ITB.Infrastructure.Repositories;

public class VeiculoRepository : IVeiculoRepository
{
    private readonly AppDbContext _context;

    public VeiculoRepository(AppDbContext context)
    {
        _context = context;
    }

    public async Task Adicionar(Veiculo veiculo)
    {
        await _context.Veiculos.AddAsync(veiculo);
        await _context.SaveChangesAsync();
    }

    public async Task Atualizar(Veiculo veiculo)
    {
        _context.Veiculos.Update(veiculo);
        await _context.SaveChangesAsync();
    }

    public async Task<IEnumerable<Veiculo>> ObterTodos()
    {
        return await _context.Veiculos
            .AsNoTracking()
            .Include(v => v.Marca)
            .ToListAsync();
    }

    public async Task<Veiculo?> ObterPorId(int id)
    {
        return await _context.Veiculos.FindAsync(id);
    }

    public async Task<bool> PlacaJaExiste(string placa, int veiculoIdIgnorado)
    {
        return await _context.Veiculos
            .IgnoreQueryFilters()
            .AnyAsync(v => v.Placa == placa && v.Id != veiculoIdIgnorado);
    }
}
```

**Passo 3: A Aplicação (Os Carteiros e o Maestro)**
Os Commands transportam os dados, e o Handler atua estritamente como o "Escrivão", levantando as provas e entregando ao Domínio.

**ITB.Application/Commands/AtualizarVeiculoCommand.cs e DesativarVeiculoCommand.cs**
***Código C#***
```
using ITB.Domain.Core.Messages.Interfaces;

namespace ITB.Application.Commands;

public class AtualizarVeiculoCommand : ICommand
{
    public int Id { get; set; }
    public string Modelo { get; set; } = string.Empty;
    public string Placa { get; set; } = string.Empty;
    public int Ano { get; set; }
    public int MarcaId { get; set; }
}

public class DesativarVeiculoCommand : ICommand
{
    public int Id { get; set; }
}
```

**ITB.Application/Handlers/AtualizarVeiculoHandler.cs**
***Código C#***
```
using System.Threading.Tasks;
using ITB.Application.Commands;
using ITB.Domain.Core.Exceptions;
using ITB.Domain.Interfaces;

namespace ITB.Application.Handlers;

public class AtualizarVeiculoHandler : IHandler<AtualizarVeiculoCommand>
{
    private readonly IVeiculoRepository _veiculoRepository;
    private readonly IMarcaRepository _marcaRepository;

    public AtualizarVeiculoHandler(IVeiculoRepository veiculoRepository,
IMarcaRepository marcaRepository)
    {
        _veiculoRepository = veiculoRepository;
        _marcaRepository = marcaRepository;
    }

    public async Task Handle(AtualizarVeiculoCommand command)
    {
        var veiculo = await _veiculoRepository.ObterPorId(command.Id);
```

```
        if (veiculo == null) throw new DomainException("Veículo não encontrado.");

        // 1. Levantamento de Fatos (O Handler é apenas o Escrivão)
        var placaDuplicada = await _veiculoRepository.PlacaJaExiste(command.Placa,
command.Id);

        bool marcaExiste = true;

        // Otimização: Evitamos ir ao banco consultar a marca se o usuário não mudou o
ID
        if (veiculo.MarcaId != command.MarcaId)
        {
            marcaExiste = await _marcaRepository.ObterPorId(command.MarcaId) != null;
        }

        // 2. Aplicação das Intenções (A Entidade é o Juiz)
        veiculo.AtualizarDados(command.Modelo, command.Ano);
        veiculo.AlterarPlaca(command.Placa, placaDuplicada);
        veiculo.AlterarMarca(command.MarcaId, marcaExiste);

        // 3. Persistência
        await _veiculoRepository.Atualizar(veiculo);
    }
}
```

**ITB.Application/Handlers/DesativarVeiculoHandler.cs**

***Código C#***

```
using System.Threading.Tasks;
using ITB.Application.Commands;
using ITB.Domain.Core.Exceptions;
using ITB.Domain.Interfaces;

namespace ITB.Application.Handlers;

public class DesativarVeiculoHandler : IHandler<DesativarVeiculoCommand>
{
    private readonly IVeiculoRepository _veiculoRepository;

    public DesativarVeiculoHandler(IVeiculoRepository veiculoRepository)
    {
        _veiculoRepository = veiculoRepository;
    }

    public async Task Handle(DesativarVeiculoCommand command)
    {
        var veiculo = await _veiculoRepository.ObterPorId(command.Id);
        if (veiculo == null) throw new DomainException("Veículo não encontrado.");

        veiculo.Desativar();
```

```
        await _veiculoRepository.Atualizar(veiculo);
    }
}
```

**Passo 4: A API (Controllers)**

**ITB.Api/Controllers/VeiculoController.cs**

***Código C#***

```
using System.Threading.Tasks;
using Microsoft.AspNetCore.Mvc;
using ITB.Application.Commands;
using ITB.Domain.Core.Messages.Interfaces;

namespace ITB.Api.Controllers;

[ApiController]
[Route("api/[controller]")]
public class VeiculoController : ControllerBase
{
    private readonly IMessageBus _bus;

    public VeiculoController(IMessageBus bus)
    {
        _bus = bus;
    }

    [HttpPut("{id}")]
    public async Task<IActionResult> Put(int id, [FromBody] AtualizarVeiculoCommand
command)
    {
        if (id != command.Id) return BadRequest(new { mensagem = "IDs divergentes." });

        await _bus.EnviarComando(command);
        return Ok(new { mensagem = "Veículo atualizado com sucesso." });
    }

    [HttpDelete("{id}")]
    public async Task<IActionResult> Delete(int id)
    {
        var command = new DesativarVeiculoCommand { Id = id };

        await _bus.EnviarComando(command);
        return Ok(new { mensagem = "Veículo desativado com sucesso." });
    }
}
```

**Mapeamento de Infraestrutura (Soft Delete)**

Para que o **Soft Delete** funcione sem que o programador tenha que lembrar de filtrar em todo SELECT do sistema, podemos usar o **Global Query Filter** na configuração do EF Core:

**Código C#**

```
// Exemplo em MarcaMapping.cs e VeiculoMapping.cs
public void Configure(EntityTypeBuilder<Marca> builder)
{
    // ... outras configurações ...

    // O EF Core vai adicionar 'WHERE Ativo = 1' em todas as consultas automaticamente
    builder.HasQueryFilter(x => x.Ativo);
}
```

🛠 **Exercício Prático: O Ciclo de Vida da Marca e Integridade Relacional**

**Cenário:** A entidade Marca precisa de um fluxo seguro de atualização (PUT) garantindo a idempotência. Além disso, a DM Card exige uma validação rigorosa para a exclusão lógica (DELETE): **"Uma marca jamais poderá ser desativada se ainda existirem veículos ativos vinculados a ela."**

**Desafio:**

1. **No Repositório de Marcas (IMarcaRepository e MarcaRepository):** Crie um método otimizado (retornando um *booleano*) que verifique no banco de dados se existem veículos ativos para um determinado MarcaId. Lembre-se de não carregar listas inteiras para a memória; use as funções assíncronas do EF Core para obter apenas o fato.
2. **Crie o Handler de Exclusão (DesativarMarcaHandler):** Utilize a técnica de "Injeção de Fatos". O Handler deve consultar o repositório para saber se existem vínculos ativos e entregar essa resposta (Sim/Não) para a Entidade Marca tomar a decisão final de desativar ou lançar uma exceção de domínio.
3. **Nas Controllers de Marca:** Crie os endpoints HTTP:
    - O método PUT, disparando o AtualizarMarcaCommand para alterar o nome da marca de forma idempotente.
    - O método DELETE, disparando o DesativarMarcaCommand e executando toda a validação de Soft Delete que você construiu.

🕵 **Dica:**

*"Lembrem-se da nossa regra de ouro: O banco de dados entrega o Fato, o Domínio toma a Decisão. Não coloquem if(temVeiculos) soltos dentro do Handler ou da Controller. A responsabilidade de barrar a desativação é estritamente do método Desativar(bool possuiVeiculosAtivos) dentro da classe Marca!"*